Software Development Project — G&G City · Lab Backlog Sprint 1

LAB P2: Project Manager + Repository — Faris Al-Hazmi

Topic: Running a Sprint with Git Branches, Pull Requests and Merges

Learning outcomes

By the end of this lab, students will be able to:

- Protect `main` and work through Pull Requests
- Merge team work and resolve conflicts
- Track the sprint and report progress

🎯 Problem Scenario

Nine people push to one repository. Without rules, someone overwrites someone else's work the night before the deadline.

- **Everybody works on a branch, you merge into `main`**
- **The two shared files `cat/cat.js` (C1 + C2) and `index.html` need extra care**

Your files: `the repository settings, the sprint board, README.md`

Required Outputs

Build:

1. **Add all members as collaborators and protect `main`**
2. **Teach the branch workflow (one branch per lab)**
3. **Review and merge Pull Requests in dependency order**
4. **Resolve conflicts, and write the sprint report**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

Model Answer:

Task 1: Repository setup

> 📝 **Note (no code needed): Settings → Collaborators**: add the 8 members (they must accept the e-mail). **Settings → Branches → Add rule** for `main`: require a pull request before merging, 1 approval.

Task 2: The branch workflow every member follows (hard part)

Send this to the group. The branch name is the lab id, so you always know whose work it is.

```
git switch main
git pull                        # always start from the latest main
git switch -c lab-c1            # one branch per lab

# ... work, then:
git add cat/
git commit -m "إضافة القط وحركتي الوقوف والمشي"
git push -u origin lab-c1       # then open a Pull Request on GitHub
```

Task 3: Merge order

> 📝 **Note (no code needed):** Merge in this order so nothing breaks: **P1** lessons.json → **S1** workshop.png → **C1** cat → **C2** skills → **C3** box → **S2** scenes → **S3** index. Before merging, pull the branch and open `index.html` with Live Server.

Task 4: Resolving a conflict (hard part)

When GitHub says *"This branch has conflicts"*, fix it locally:

```
git switch lab-c2
git pull origin main            # Git marks the conflict inside the file:

# <<<<<<< HEAD
#   (C2's version)
# =======
#   (main's version)
# >>>>>>> main

# keep both parts if both are needed, delete the 3 marker lines, then:
git add cat/cat.js
git commit -m "حل تعارض cat.js"
git push
```

> 📝 **Note (no code needed):** Sprint report: for each lab, Done / In progress / Blocked + the reason. Attach a screenshot of the game at the end of the sprint.